Rhilo Sotto
5/7/2025
COMPE 470L

Final Report

Introduction

For this project, I designed and implemented a digital slot machine using Verilog and a 4-digit seven-segment display on an FPGA board. The LEDs display the user's total score, simulate reels spinning, and show the symbol they land on. The LEDs update to using digit multiplexing to get all 4 LEDs to show up without flickering. There is a button to spin the wheel, as well as a button to reset the machine. I used a BCD to Binary conversion module I found online to get the score to display on the LED, multiplexed display control, and a state machine to switch between different internal states.

Verilog Code

Top Module

```
module top(
    input CLK_100MHZ,
    input RST,
    input BTN,

    output [3:0] digit,
    output [6:0] seg
    );

    wire [15:0] Score;

    wire done_r1, done_r2, done_r3, done_r4;
    wire done_ns1, done_ns2, done_ns3, done_ns4;
    wire done_sc;
    wire done_bc;
    wire done_btbcd;
    wire valid;

    wire [7:0] num_1, num_2, num_3, num_4;
    wire [1:0] symbol_1, symbol_2, symbol_3, symbol_4;
```

```
    wire [3:0] score_sc;

    // score digits
    wire [3:0] score_thousands, score_hundreds, score_tens, score_ones;


//    wire BTN_db, RST_db;
//    // BUTTON DEBOUNCING
//    ButtonDebounce BDS(.CLK(CLK_100MHZ), .BTN_in(BTN), .BTN_out(BTN_db));
//    ButtonDebounce BDR(.CLK(CLK_100MHZ), .BTN_in(RST), .BTN_out(RST_db));


    wire CLK_1S;
    // CLOCK SCALER FOR REELS PROCESS (SLOW FOR 10 CLOCK CYCLES)
    // 100MHZ / 10000000 = 10Hz = 0.1s
    ClockScaler #(.ClockDivider(10000000)) CS1S(
        .CLK(CLK_100MHZ),
        .RST(RST),
        .CLK_scaled(CLK_1S)
    );

    wire CLK_1MS;
    // CLOCK SCALER FOR LED DIGITS
    // 100MHZ / 100000 = 1kHz = 1ms
    ClockScaler #(.ClockDivider(100000)) CS5MS(
        .CLK(CLK_100MHZ),
        .RST(RST),
        .CLK_scaled(CLK_1MS)
    );

    Reel #(.SEED(8'b10101000)) R1(.CLK(CLK_1S), .RST(RST), .spin(valid), .num(num_1),
.done(done_r1));
    Reel #(.SEED(8'b10101010)) R2(.CLK(CLK_1S), .RST(RST), .spin(valid), .num(num_2),
.done(done_r2));
    Reel #(.SEED(8'b11101000)) R3(.CLK(CLK_1S), .RST(RST), .spin(valid), .num(num_3),
.done(done_r3));
    Reel #(.SEED(8'b00101001)) R4(.CLK(CLK_1S), .RST(RST), .spin(valid), .num(num_4),
.done(done_r4));


    NumToSymbol NS1(.CLK(CLK_1S), .convert(done_r1), .num(num_1), .symbol(symbol_1),
.done(done_ns1));
    NumToSymbol NS2(.CLK(CLK_1S), .convert(done_r2), .num(num_2), .symbol(symbol_2),
.done(done_ns2));
```

```
   NumToSymbol NS3(.CLK(CLK_1S), .convert(done_r3), .num(num_3), .symbol(symbol_3),
.done(done_ns3));
   NumToSymbol NS4(.CLK(CLK_1S), .convert(done_r4), .num(num_4), .symbol(symbol_4),
.done(done_ns4));


   ScoreCalculator SC(
       .CLK(CLK_1S),
       .calculate(done_ns4),
       .Symbol_1(symbol_1), .Symbol_2(symbol_2), .Symbol_3(symbol_3),
.Symbol_4(symbol_4),
       .Score(score_sc),
       .done(done_sc)
       );


   BetController BC(
       .CLK(CLK_1S),
       .RST(RST),
       .BTN(BTN),
       .calculate(done_sc),
       .base(score_sc),
       .valid(valid),
       .Score(Score),
       .done(done_bc)
   );

   // MODULE TO CONVERT BINARY SCORE TO DECIMAL DIGITS
   BinaryToBCD #(.INPUT_WIDTH(16), .DECIMAL_DIGITS(4)) BtBCD(
       .CLK(CLK_100MHZ),
       .score(Score),
       .convert(done_bc),
       .Done(done_btbcd),
       .score_thousands(score_thousands), .score_hundreds(score_hundreds),
.score_tens(score_tens), .score_ones(score_ones)
   );


   LEDDisplay LD(
       .CLK(CLK_100MHZ), .CLK_display(CLK_1MS),
       .RST(RST),

       .spin(BTN),
       .num_1(num_1), .num_2(num_2), .num_3(num_3), .num_4(num_4),
```

```
        .convert(done_r4),
        .symbol_1(symbol_1), .symbol_2(symbol_2), .symbol_3(symbol_3), .symbol_4(symbol_4),

        .display(done_btbcd), // done signal from binarytobcd
        // each digit of total score
        .score_thousands(score_thousands), .score_hundreds(score_hundreds),
.score_tens(score_tens), .score_ones(score_ones),

        .digit(digit),
        .seg(seg)
    );


endmodule
```

Clock Scaler Module

```
module ClockScaler #(parameter ClockDivider = 1000) (
    input CLK,
    input RST,
    output reg CLK_scaled = 0
);

    reg [$clog2(ClockDivider)-1:0] ClockCounter = 0;

    always @(posedge CLK or posedge RST) begin
        if (RST) begin
            ClockCounter <= 0;
        end
        else begin
            if (ClockCounter < ClockDivider) begin
                ClockCounter <= ClockCounter + 1;
            end
            else begin
                ClockCounter <= 0;
                CLK_scaled <= ~CLK_scaled;
            end
        end
    end
```

```
endmodule
```

Reel Module

```
module Reel #(parameter SEED = 8'b00000001) (
    input CLK,
    input RST,
    input spin,
    output reg [7:0] num = SEED,
    output reg done = 0
    );

    // States
    localparam Idle = 2'b00, Spinning = 2'b01, Done = 2'b10;
    reg [1:0] state = Idle;
    // how many times to cycle before stopping
    reg [3:0] spin_count;


    always @(posedge CLK or posedge RST) begin
        if(RST) begin // reset signal
            state <= Idle;
            done <= 0;
            spin_count <= 4'b0000;
            num <= SEED;
        end
        else begin
            case(state)
                Idle: begin
                    done <= 0;
                    if(spin) begin
                        state <= Spinning;
                        spin_count <= 4'b0000;
                    end
                end
                Spinning: begin
                    if(spin_count < 4'b0100) begin // 8 Bit LFSR with tap bits at 8, 6, 5, 4 for maximal
length
```

```
                num <= {num[6], num[5], num[4], num[3], num[2], num[1], num[0],
num[7]^num[5]^num[4]^num[3]};
                spin_count <= spin_count + 1;
            end
            else begin
                state <= Idle;
                done <= 1;
            end
        end

    endcase
  end
end

endmodule
```

NumberToSymbol Module

```
module NumToSymbol(
    input CLK,
    input convert,
    input [7:0] num,
    output reg [1:0] symbol,
    output reg done
    );


    // 4 symbols with varying probabilities
    localparam A = 2'b00, B = 2'b01, C = 2'b10, D = 2'b11;

    // 256 possible nums
    always @(posedge CLK) begin
        if(convert) begin
            if(num < 128) begin // 50%
                symbol <= A;
            end
            else if(num < 128 + 64) begin // 25%
                symbol <= B;
            end
            else if(num < 128 + 64 + 32 + 16) begin // 12.5% + 6.25%
                symbol <= C;
```

```
            end
            else begin // 6.25%
              symbol <= D;
            end
            done <= 1;
        end
        else begin
          done <= 0;
        end
    end



endmodule
```

ScoreCalculator Module

```
module ScoreCalculator(
    input CLK,
    input calculate,
    input [1:0] Symbol_1, Symbol_2, Symbol_3, Symbol_4,
    output reg [3:0] Score,
    output reg done
    );

    // 4 symbols with varying probabilities
    localparam A = 2'b00, B = 2'b01, C = 2'b10, D = 2'b11;

    always @(posedge CLK) begin
        if(calculate) begin
            if ((Symbol_1 == Symbol_2) && (Symbol_2 == Symbol_3) && (Symbol_3 == Symbol_4))
begin
                case(Symbol_4) // all four are the same, payout based on rarity
                A: Score <= 1;
                B: Score <= 3;
                C: Score <= 5;
                D: Score <= 15;
                endcase
            end
            else begin
```

```
            Score <= 0; // at least one reel is different, lose
        end
        done <= 1;
    end
    else begin
        done <= 0;
    end
  end

endmodule
```

BetController Module

```
module BetController(
  input CLK,
  input RST,
  input BTN,
  input calculate,

  input [3:0] base,

  output reg valid,
  output reg [15:0] Score = 100,

  output reg done
  );

  // States
  localparam Idle = 2'b00, Spinning = 2'b01, Done = 2'b10;
  reg [1:0] state = Idle;

  // bet amount, always 1
  reg [3:0] bet = 1;

  always @(posedge CLK or posedge RST) begin
    if(RST) begin // reset signal
        Score <= 100;
        state <= Idle;
        bet <= 1;
        valid <= 0;
        done <= 0;
```

```
        end
        else begin
            case(state)
                Idle: begin // waiting for inputs
                    done <= 0;
                    valid <= 0;
                    if(BTN) begin // check if user has enough score to spin
                        if(Score < bet) begin
                            valid <= 0;
                        end
                        else begin
                            Score <= Score - bet;
                            valid <= 1;
                            state <= Spinning;
                        end
                    end
                end
                Spinning: begin
                    valid <= 0; // wait for base score to be calculated by scorecalculator
                    if(calculate) begin
                        Score <= (Score + bet * base > 9999) ? 9999 : Score + bet * base; // cap score to
9999
                        done <= 1;
                        state <= Idle;
                    end
                end

            endcase
        end
    end

endmodule
```

BCDToBinary Module

```
module BinaryToBCD
  #(parameter INPUT_WIDTH = 16,
    parameter DECIMAL_DIGITS = 4)
  (
   input CLK,
   input [INPUT_WIDTH-1:0] score,
   input convert,
   //
//   output [DECIMAL_DIGITS*4-1:0] o_BCD,
   output reg Done,

   output reg [3:0] score_thousands, score_hundreds, score_tens, score_ones

   );

  localparam Idle = 3'b000, Shift = 3'b001, Check_shift_index = 3'b010,
  Add = 3'b011, Check_digit_index = 3'b100, Complete = 3'b101;

  reg [2:0] State = Idle;

  // The vector that contains the output BCD
  reg [DECIMAL_DIGITS*4-1:0] r_BCD = 0;

  // The vector that contains the input binary value being Shifted.
  reg [INPUT_WIDTH-1:0]      r_Binary = 0;

  // Keeps track of which Decimal Digit we are indexing
  reg [DECIMAL_DIGITS-1:0]   r_Digit_Index = 0;

  // Keeps track of which loop iteration we are on.
  // Number of loops performed = INPUT_WIDTH
  reg [7:0]                  r_Loop_Count = 0;

  wire [3:0]                 w_BCD_Digit;

  always @(posedge CLK)
    begin

      case (State)

        // Stay in this state until convert comes along
        Idle :
```

```
  begin
    Done <= 1'b0;

    if (convert == 1'b1)
      begin
        r_Binary  <= score;
        State <= Shift;
        r_BCD     <= 0;
      end
    else
      State <= Idle;
  end


// Always Shift the BCD Vector until we have Shifted all bits through
// Shift the most significant bit of r_Binary into r_BCD lowest bit.
Shift :
  begin
    r_BCD     <= r_BCD << 1;
    r_BCD[0]  <= r_Binary[INPUT_WIDTH-1];
    r_Binary  <= r_Binary << 1;
    State <= Check_shift_index;
  end


// Check if we are Complete with Shifting in r_Binary vector
Check_shift_index :
  begin
    if (r_Loop_Count == INPUT_WIDTH-1)
      begin
        r_Loop_Count <= 0;
        State    <= Complete;
      end
    else
      begin
        r_Loop_Count <= r_Loop_Count + 1;
        State    <= Add;
      end
  end

// Break down each BCD Digit individually. Check them one-by-one to
// see if they are greater than 4. If they are, increment by 3.
// Put the result back into r_BCD Vector.
Add :
```

```
      begin
        if (w_BCD_Digit > 4)
          begin
            r_BCD[(r_Digit_Index*4)+:4] <= w_BCD_Digit + 3;
          end

        State <= Check_digit_index;
      end


    // Check if we are Complete incrementing all of the BCD Digits
    Check_digit_index :
      begin
        if (r_Digit_Index == DECIMAL_DIGITS-1)
          begin
            r_Digit_Index <= 0;
            State     <= Shift;
          end
        else
          begin
            r_Digit_Index <= r_Digit_Index + 1;
            State     <= Add;
          end
      end


    Complete :
      begin
        Done      <= 1'b1;
        State <= Idle;


        score_thousands <= r_BCD[15:12];
        score_hundreds <= r_BCD[11:8];
        score_tens <= r_BCD[7:4];
        score_ones <= r_BCD[3:0];
      end


    default :
      State <= Idle;

  endcase
end // always @ (posedge CLK)
```

```
  assign w_BCD_Digit = r_BCD[r_Digit_Index*4 +: 4];

//  assign o_BCD = r_BCD;

endmodule // Binary_to_BCD
```

LED Display Module

```
module LEDDisplay(
    input CLK, CLK_display,
    input RST,

    input spin,
    input [7:0] num_1, num_2, num_3, num_4,

    input convert,
    input [1:0] symbol_1, symbol_2, symbol_3, symbol_4,

    input display, // bcd conversion signal from binarytobcd
    // each digit of total score
    input [3:0] score_thousands, score_hundreds, score_tens, score_ones,


    output reg [3:0] digit,
    output reg [6:0] seg
    );


    // States
    localparam Idle = 2'b00, Spinning = 2'b01, Symbol = 2'b10;
    reg [1:0] state = Idle;

    // 4 symbols with varying probabilities
    localparam A = 2'b00, B = 2'b01, C = 2'b10, D = 2'b11;

    reg [1:0] digit_sel = 2'b00;
    reg[3:0] bcd_in = 0;

    // switch states and digits to be displayed
    always @(posedge CLK or posedge RST) begin
        if(RST) begin // reset signal
```

```
        state <= Idle;
    end
    else begin
        case(state)
            Idle: begin
                if(spin) begin
                    state <= Spinning;
                end
            end
            Spinning: begin
                if(convert) begin
                    state <= Symbol;
                end
            end
            Symbol: begin
                if(display) begin
                    state <= Idle;
                end
            end

        endcase
    end
end


// cycle digits
always @(posedge CLK_display or posedge RST) begin
    if (RST)
        digit_sel <= 2'b00;
    else
        digit_sel <= digit_sel + 1;
end


// process digits depending on current state
always @(posedge CLK) begin
    case(state)
        Idle: begin
        // each state has a different input + map
        case(digit_sel)
            2'b00: begin digit = 4'b0111; bcd_in = score_thousands; end
            2'b01: begin digit = 4'b1011; bcd_in = score_hundreds; end
            2'b10: begin digit = 4'b1101; bcd_in = score_tens; end
            2'b11: begin digit = 4'b1110; bcd_in = score_ones; end
```

```
endcase
// decimals showing
case(bcd_in)
   4'h0: seg = 7'b1000000; // 0
   4'h1: seg = 7'b1111001; // 1
   4'h2: seg = 7'b0100100; // 2
   4'h3: seg = 7'b0110000; // 3
   4'h4: seg = 7'b0011001; // 4
   4'h5: seg = 7'b0010010; // 5
   4'h6: seg = 7'b0000010; // 6
   4'h7: seg = 7'b1111000; // 7
   4'h8: seg = 7'b0000000; // 8
   4'h9: seg = 7'b0010000; // 9
   default: seg = 7'b1111111; // All segments off
endcase

end
Spinning: begin
case(digit_sel)
   2'b00: begin digit = 4'b0111; bcd_in = num_1; end
   2'b01: begin digit = 4'b1011; bcd_in = num_2; end
   2'b10: begin digit = 4'b1101; bcd_in = num_3; end
   2'b11: begin digit = 4'b1110; bcd_in = num_4; end
endcase
// bcd_in is random cycling, displays the random cycling
seg = bcd_in;
end
Symbol: begin
case(digit_sel)
   2'b00: begin digit = 4'b0111; bcd_in = symbol_1; end
   2'b01: begin digit = 4'b1011; bcd_in = symbol_2; end
   2'b10: begin digit = 4'b1101; bcd_in = symbol_3; end
   2'b11: begin digit = 4'b1110; bcd_in = symbol_4; end
endcase
// symbols to show on 7 segment display
case(bcd_in)
   A: seg = 7'b1111111; // 0 FOR NOW
   B: seg = 7'b1111110; // 1
   C: seg = 7'b1111100; // 2
   D: seg = 7'b1111000; // 3

   default: seg = 7'b1111111; // All segments off
endcase
end
```

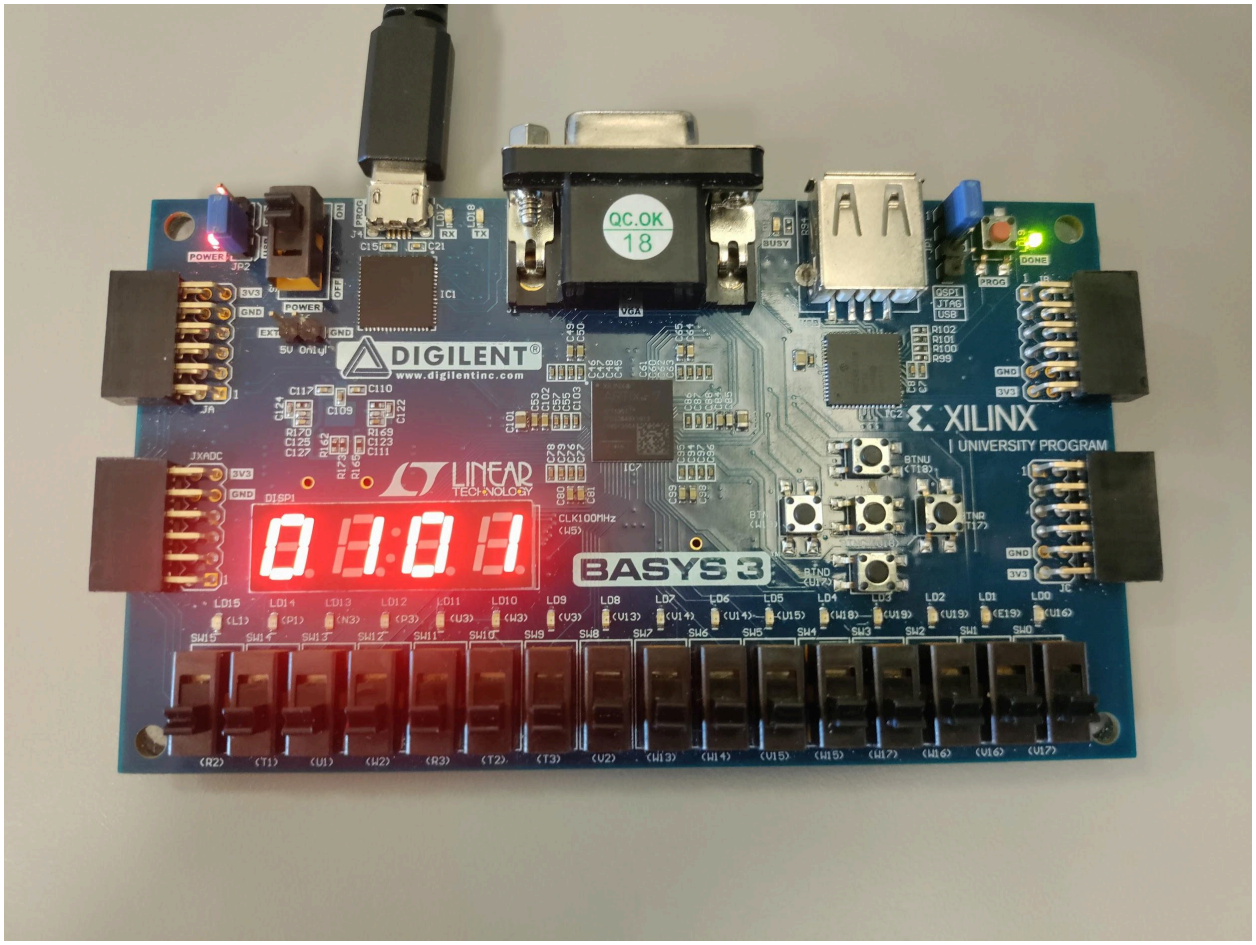
```
        endcase
    end

endmodule
```

Constraint File

```
# Clock signal
set_property PACKAGE_PIN W5 [get_ports CLK_100MHZ]
   set_property IOSTANDARD LVCMOS33 [get_ports CLK_100MHZ]
   create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports
CLK_100MHZ]


#7 segment display
set_property PACKAGE_PIN W7 [get_ports {seg[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]
set_property PACKAGE_PIN W6 [get_ports {seg[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
set_property PACKAGE_PIN U8 [get_ports {seg[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
set_property PACKAGE_PIN V8 [get_ports {seg[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
set_property PACKAGE_PIN U5 [get_ports {seg[4]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]
set_property PACKAGE_PIN V5 [get_ports {seg[5]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
set_property PACKAGE_PIN U7 [get_ports {seg[6]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]

set_property PACKAGE_PIN U2 [get_ports {digit[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {digit[0]}]
set_property PACKAGE_PIN U4 [get_ports {digit[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {digit[1]}]
set_property PACKAGE_PIN V4 [get_ports {digit[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {digit[2]}]
set_property PACKAGE_PIN W4 [get_ports {digit[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {digit[3]}]


#Buttons
set_property PACKAGE_PIN W19 [get_ports {BTN}]
   set_property IOSTANDARD LVCMOS33 [get_ports {BTN}]
set_property PACKAGE_PIN U18 [get_ports {RST}]
   set_property IOSTANDARD LVCMOS33 [get_ports {RST}]
```

Results

Video showcasing functionality: [COMPE470L Final Project Video.mp4]



Current score being displayed

Conclusion

For this project, I successfully implemented a functional digital slot machine on an FPGA board. I utilized a combination of Verilog modules to control reel spinning, number to symbol converting, score calculating, bet controlling, and LED displaying. If I had more time, I would've tried to implement changing betting amounts, and making the timing between the reels spinning, symbols displaying, and score more obvious for the user.